

Практика 11. Стандартные монады

Андрей Стоян, Илья Колегов, Дмитрий Халанский

MSE ITMO

осень 2025

В предыдущих сериях

- ▶ Классы типов `Functor`, `Applicative` и `Monad`
- ▶ Стандартные монады `Identity`, `Maybe` и `[]`
- + Стандартные монады `Reader`, `Writer`, `State`, `IO`, `Except`

Содержание

1. Монада Writer
2. Монада Reader
3. Монада State
4. Монада IO
5. Материалы

Содержание

1. Монада Writer
2. Монада Reader
3. Монада State
4. Монада IO
5. Материалы

Монада Writer

```
newtype Writer w a = Writer { runWriter :: (w, a) }
```

todo Реализуйте `Functor` для `Writer`

todo Реализуйте `Applicative` для `Writer`

todo Реализуйте `Monad` для `Writer`

todo Реализуйте следующий код с помощью монады `Writer`

```
f a b =
  let (w1, x) = g a in
  let w2 = "ou" in
  let (w3, y) = h b in
  (w1 <> w2 <> w3, x + y)
```

```
f a b = runWriter do
  x <- g' a
  tell "ou"
  y <- h' b
  pure (x + y)
```

```
tell :: w -> Writer w ()
tell w = Writer (w, ())
```

Содержание

1. Монада Writer
2. Монада Reader
3. Монада State
4. Монада IO
5. Материалы

Монада Reader

```
newtype Reader e a = Reader { runReader e -> a }
```

todo Реализуйте **Functor** для **Reader**

todo Реализуйте **Applicative** для **Reader**

todo Реализуйте **Monad** для **Reader**

todo Реализуйте следующий код с помощью монады **Reader**

```
f :: Int -> Env -> Int
f a env =
  let x = g a env in
  let y = intField env in
  let z = h env in
  x + y + z
```

```
f :: Int -> Env -> Int
f a = runReader do
  x <- g' a
  y <- asks intField
  z <- h'
  pure (x + y + z)
```

```
asks :: (e -> e') -> Reader e e'
asks = Reader
```

Мотивация Reader

e.g. Конфигурация программы, нужна большому количеству кода

e.g. По некоторой функциональности нужно абстрагироваться

- ▶ Возможности подмены реализации,

e.g. Для тестирования

- ▶ Вариант: singleton object — глобальный объект, который доступен отовсюду

- + В тупом варианте очень просто и везде работает (почти)

- Тяжело подменить — минус тестируемость

- Нет гранулярного контроля доступа к глобальному объекту

- ▶ Вариант: передача везде доп. аргумента — окружения (коробочка со значениями и функциями)

- + Зависимость записана в сигнатурах

- + Просто и работает

- Передача всегда доп. аргумента засоряет код

- ⇒ Нужен механизм неявной передачи некоторых аргументов

Inversion of control & dependency injection (☀)

△ Inversion of control — абстрагирование по реализации

- ▶ Вместо статического вызова конкретной функции, принимаем её как зависимость
- + Реализации можно незаметно подменять
- Дополнительный уровень косвенности для вызовов — дороже статического вызова

△ Dependency injection — механизм распространения функциональности по коду

- ▶ Передавать реализации самостоятельно — утомительно
- + Можно получить заданную реализацию в произвольной точке кода
- Часто реализуется чёрной магией, а не средствами языка

Можно обойтись монадой **Reader**!

```
data Auth { check :: UserId -> Bool } -- Сервис (набор функций) авторизации.
```

```
doSmtH :: UserId -> Reader Auth Int
```

```
doSmtH uid = do
```

```
  b <- asks check uid -- Достаём из окружения сервис авторизации и используем.
  ...                -- Логика, зависящая от авторизованности пользователя.
```

Содержание

1. Монада Writer
2. Монада Reader
3. Монада State
4. Монада IO
5. Материалы

Персистентные структуры данных

Мутабельная структура данных

```
val xs = MutableList<Int>()
xs.add(42)
xs.add(4)
val res = xs.pop()
...
```

Персистентная структура данных

```
let xs      = emptyList
  xs'      = cons 42 xs
  xs''     = cons 4  xs'
  (xs''', res) = uncons xs'''
...
```

- Перепишем типы операций

```
cons    :: Int -> List Int -> (List Int, () )
uncons  ::      List Int -> (List Int, Int)
```

- Вынесем общую часть в отдельный тип

```
newtype State s a = State { runState :: s -> (s, a) }
cons'    :: Int -> State (List Int) ()
uncons'  ::      State (List Int) Int
```

- Монада **State** — мутабельность как интерфейс над персистентностью

Монада State

```
newtype State s a = State { runState :: s -> (s, a) }
```

todo Реализуйте **Functor** для **State**

todo Реализуйте **Applicative** для **State**

todo Реализуйте **Monad** для **State**

todo Реализуйте следующий код с помощью монады **State**

```
f :: Int -> Int -> (Int, ())
f a st1 =
  let (st2, b) = g a st1 in
  let st3 = st2 + 1 in
  (st3, ())
```

```
f :: Int -> Int -> (Int, ())
f x = runState do
  b <- g' a
  st <- get
  put (st + 1)
get :: State s s
get = State \s -> (s, s)
put :: s -> State s ()
put s' = State \s -> (s', ())
```

Содержание

1. Монада Writer
2. Монада Reader
3. Монада State
4. Монада IO
5. Материалы

IO как State

```
type IO a = State RealWorld a -- RealWorld -> (a, RealWorld)
```

NB Это только mind-model, а не правда

```
-- | @RealWorld@ is deeply magical. It is /primitive/, but it is not
--      /unlifted/ (hence @ptrArg@). We never manipulate values of type
--      @RealWorld@; it's only used in the type system, to parameterise @State#@.
data RealWorld
```

- ▶ Каждая программа меняет мир, меняйте мир к лучшему!

```
▶ main :: IO () -- RealWorld -> ((), RealWorld)
```

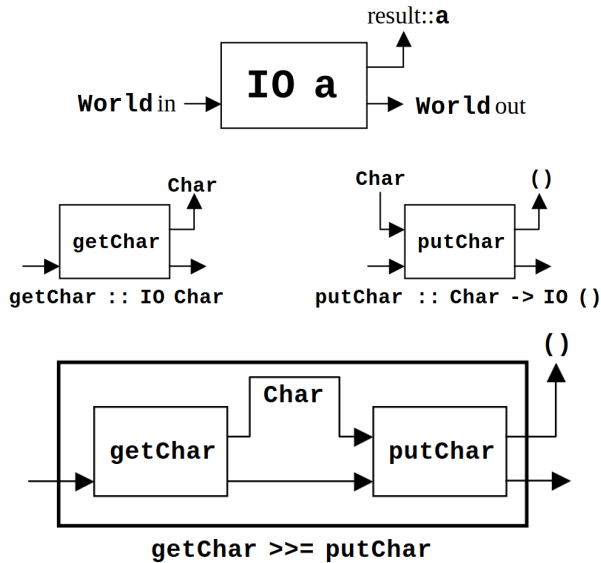
- ▶ Эффект IO выполняется только когда он повлиял на результирующий RealWorld

```
- main = let p = print 42 in pure ()
+ main = let p = print 42 in p *> pure ()
```

- ▶ RealWorld — интерфейс ко всему могуществу операционной системы

e.g. Мутабельная память, потоки, синхронизация...

Картинки



Чистый язык и “грязный” runtime (☁️)

Runtime

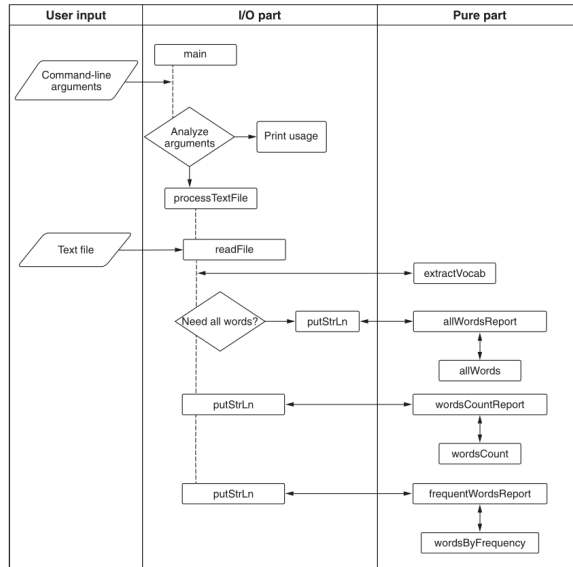
Два значения, нас интересует второе:

- △ Промежуток времени, в который программа выполняется
 - △ Код, поставляемый вместе с ЯП, который существует в адресном пространстве программ на этом ЯП и помогает им исполняться
 - e.g. garbage collector (GC), entry point — код до `main...`
-
- ▶ Незапущенная программа на Haskell — чистая функция, реальность — персистентная структура данных
 - ▶ Запуск программы передаёт управление entry point, которая конструирует RealWorld и передаёт написанному “чистому” `main`
 - ▶ Haskell остаётся чистым языком программирования, это только runtime грязный!

Контроль чистоты вычислений

- ▶ Из **IO** нельзя достать значение кроме как в `bind`
- ▶ Вычисление, один раз ставшее “грязным”, “заражает” все зависящие вычисления
- ▶ Отслеживаем в **IO**, какие вычисления завязаны на кошмар реальности
- + Типы отделяют чистые вычисления от “грязных”
- + Нужно максимизировать чистую часть программы, локализуя “грязную”
- + На побочные эффекты распространяются алгебраические законы монад

Минимизируем “грязную” часть программы¹



¹Картинка из книги Haskell in Depth Виталия Брагилевского.

Hello world

```
main :: IO ()  
main = putStrLn "Hello world!"
```



Содержание

1. Монада Writer
2. Монада Reader
3. Монада State
4. Монада IO
5. Материалы

Что посмотреть в транспорте

- ▶ (youtube) Nickolas Wu — The Evolution of Effects
- ▶ (youtube) Kris Jenkins — Side-Effects Are The Complexity Iceberg

- ▶ https://wiki.haskell.org/IO_inside
- ▶ Jones, S.P., 2001. Tackling the awkward squad: monadic input/output, concurrency, exceptions, and foreign-language calls in Haskell. NATO SCIENCE SERIES SUB SERIES III COMPUTER AND SYSTEMS SCIENCES, 180, pp.47-96.
- ▶ Marlow, Simon. "Parallel and concurrent programming in Haskell." *Central European Functional Programming School*. Berlin, Heidelberg: Springer Berlin Heidelberg, 2011. 339-401.